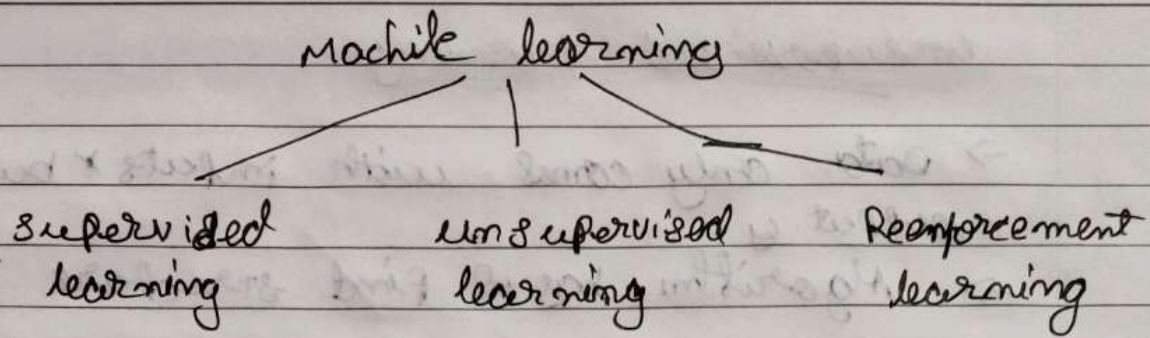


INDEX

1-2	Intro to ML
3-10	Linear regression, cost function, gradient descent
11-14	logistic regression
15-17	Regularization
18-22	INTRO TO NN
22-28	forward prop, backward prop, computation graph, Chain rule
29-30	Diagnosing bias, variance
31-32	precision, recall, F1 score, confusion matrix
33-35	Decision tree
36-37	Random forest, XG boosting
39	intro to unsupervised learning, clustering
40-42	K-means
43-45	Anomaly detection.



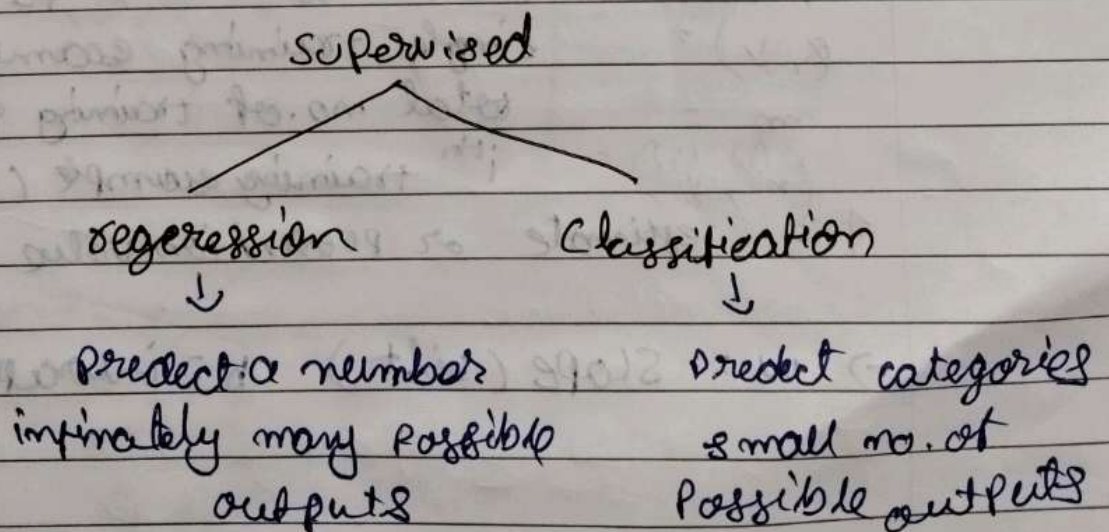
supervised learning:-

It is a type of ML model where model is trained on labeled dataset, meaning each input (x) comes with correct output (y).

the algorithm learns the mapping from input to output so when it sees new input it predicts output.

Ex:

input (x)	output (y)	Application
email	spam? (0/1)	spam filtering
audio	text transcription	speech recog



unsupervised learning

- data only comes with inputs x but not output y
- Algorithm has to find structure in a data

Clustering:-

grouping similar data points together

Anomaly detection:-

find unusual data points

Dimensionality reduction:-

compress data using fewer numbers.

terminology:-

- x = "input" variable $\rightarrow$ feature
- y = "output" variable $\rightarrow$ target
- (x, y) = single training example
- m = total no. of training examples
- $(x^{(i)}, y^{(i)})$ = i^{th} training example ($1^{\text{st}}, 2^{\text{nd}}, 3^{\text{rd}} \dots$)
- $\hat{y}$ = estimate or prediction value

$\rightarrow w$ = Slope (tilt) b = intercept (starting point)

Linear regression

training set

features
targets

learning algorithm

x
in

f (function) or (hypothesis)
out

univariate linear regression $\rightarrow$ one variable linear reg

$$f_{w,b}(x) = wx + b$$

w, b = parameters (or) coefficients (or) weights

3D, what are w, b ?

assume $f_{w,b}(x) = f(x)$

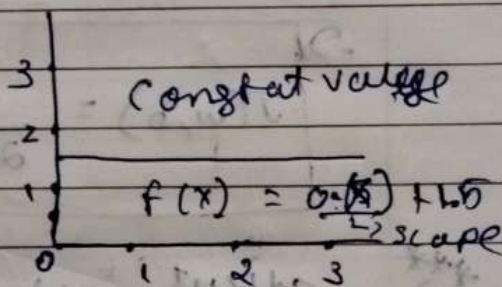
$\rightarrow$ let's take an ex:

1. let $w = 0, b = 1.5$

$$\Rightarrow f(x) = wx + b$$

$$f(x) = 0(x) + 1.5$$

$$f(x) = 1.5$$



$\rightarrow$ 2. let $w = 0.5, b = 0$

$$f(x) = wx + b$$

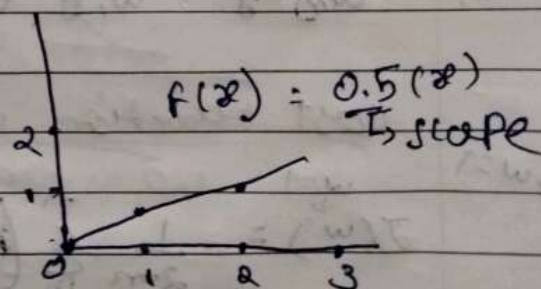
$$= 0.5(x) + 0$$

$$= 0.5(x)$$

$$\text{If } x=0, y=0$$

$$1 \quad 0.5$$

$$2 \quad 1$$



cost function or loss function:-

→ it's a math function that measures how wrong or bad is your predictions are compared to actual value

→ cost function takes predicted output ($\hat{y}$) and target (y) calculates the difference (error).

→ less the value better the prediction

$$\therefore \text{Cost function } \left(\sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})^2 \right) \cdot \frac{1}{2m}$$

$$J(w, b) = \frac{1}{2m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})^2$$

↖ Squared error cost fun

$$\Rightarrow \hat{y} = f_{w,b}(x^{(i)})$$

$$\Rightarrow J(w, b) = \frac{1}{2m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)})^2$$

→ adjust w, b such that cost function is low

cost function intuition:- if $b=0$

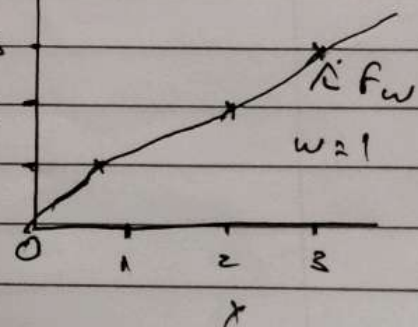
for $w=1$

$$J(w) = \frac{1}{2m} \sum_{i=1}^m (w \cdot x^{(i)} - y^{(i)})^2$$

$$= \frac{1}{2m} (w \cdot 1 - 0)^2$$

for $i=1, 2, 3$

$$J(w) = \frac{1}{2m} (0^2 + 0^2 + 0^2) = 0$$

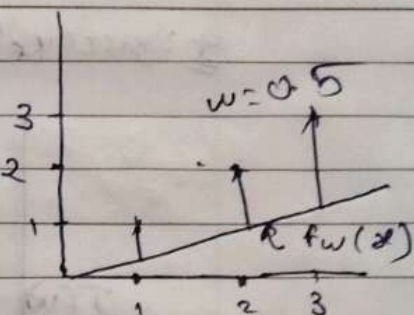


for $w = 0.5$

$$J(0.5) =$$

$$\frac{1}{2m} [0.5 - 1)^2 + (2)^2 + (4.5 - 3)^2]$$

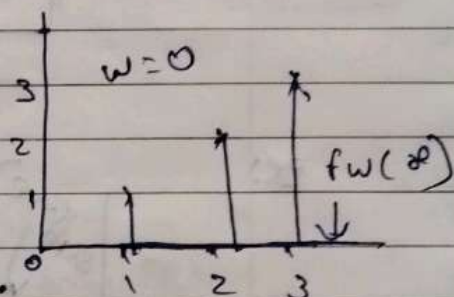
$$J(w) = \frac{1}{2(3)} [3.5] = 0.58$$



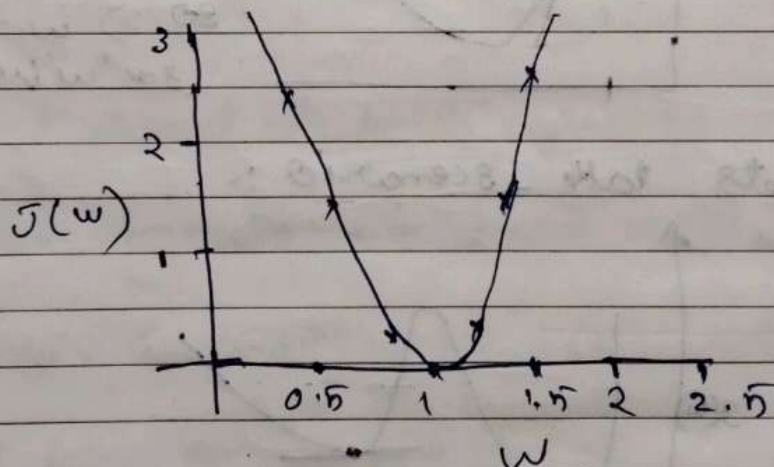
for $w = 0$

$$= \frac{1}{2(3)} [1^2 + 2^2 + 3^2]$$

$$J(w) = \frac{1}{6} (14) = 2.3$$



if we plot all $J(w)$ vs w



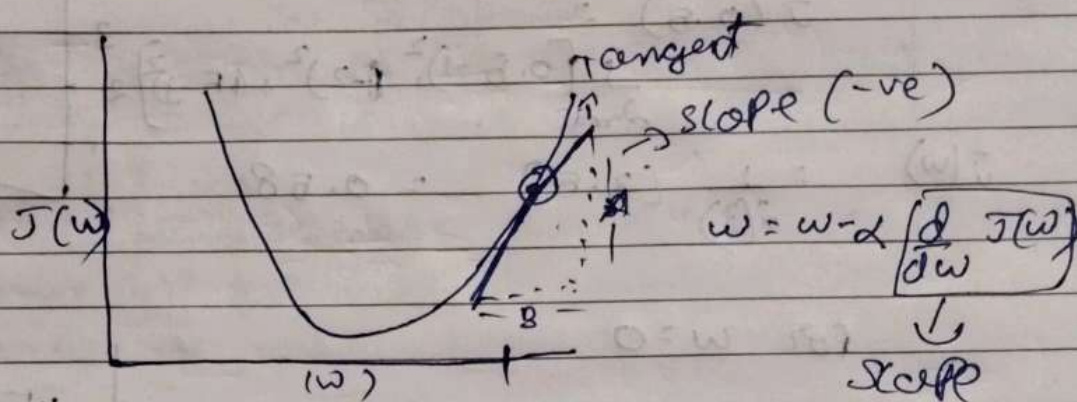
Gradient descent algorithm:-

→ used to find min value of cost functions
best parameters

$$w = w - \alpha \cdot \frac{\partial}{\partial w} J(w, b)$$

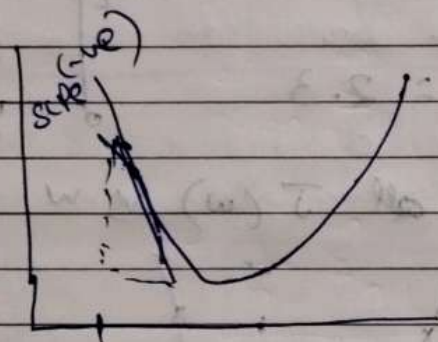
$$b = b - \alpha \cdot \frac{\partial}{\partial b} J(w, b)$$

Ex: Let's take a sample $J(w)$ (b.20).



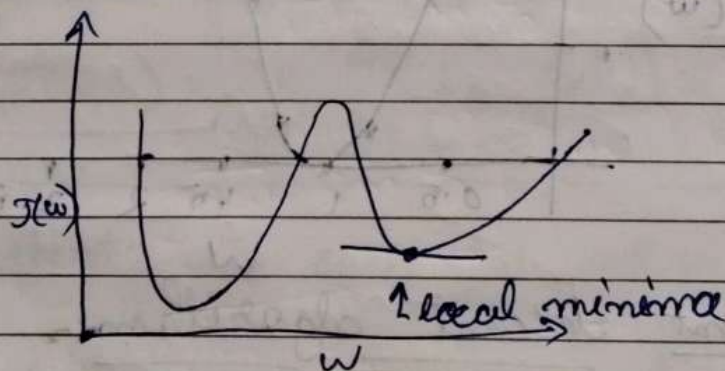
$\Rightarrow w = w - \alpha$ (+ve no.)
so w will decrease

Ex:



$\Rightarrow w = w - \alpha$ (-ve) no.
so $\Rightarrow w = w + \alpha$ (no.)
so w will increase

Let's take scenario :-



$\Rightarrow$ we are already in local minima, so slope $\rightarrow$
 $\Rightarrow \frac{dJ}{dw} = 0$

$$\Rightarrow w = w - \alpha(0)$$

$$\Rightarrow w = w$$

"we can't move further"

Can we reach local min with fixed α ?

→ yes

$$w = w - \alpha \frac{d}{dw} J(w)$$

1. $\rightarrow$ big value

2. $\rightarrow$ not as large as before

3. small

4. smaller

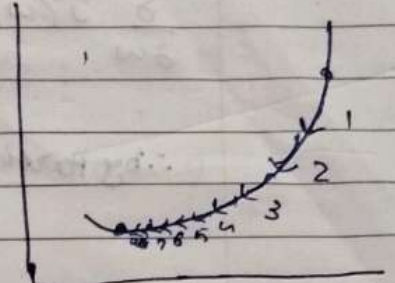
go on

$\therefore$ Near local minimum:-

- Derivative becomes smaller

- update steps become smaller

so we can reach ^{min} learning value without decreasing α



Gradient descent for linear regression :-

so all formulas we know until now are

1. Linear reg. model: $f_{wb}(x) = wx + b$

2. cost function = $J(w, b) = \frac{1}{2m} \sum_{i=1}^m (f_{wb}(x^{(i)}) - y^{(i)})^2$

3. gradient descent: $w = w - \alpha \frac{\partial}{\partial w} J(w, b)$

$$\frac{1}{m} \sum_{i=1}^m (f_{wb}(x^{(i)}) - y^{(i)}) x^{(i)}$$

$$b = b - \alpha \frac{\partial}{\partial b} J(w, b)$$

$$\frac{1}{m} \sum_{i=1}^m (f_{wb}(x^{(i)}) - y^{(i)})$$

Derivation proof for "Grad Desc"

$$\frac{\partial}{\partial w} J(w, b) = \frac{\partial}{\partial w} \left(\frac{1}{2m} \sum_{i=0}^m \left(f_{wb}(x^{(i)}) - y^{(i)} \right)^2 \right)$$

∴ by partial deriv $\Rightarrow \frac{\partial}{\partial w} \left(\frac{1}{2m} \sum_{i=0}^m \left((wx^{(i)} + b) - y^{(i)} \right)^2 \right)$

$$\rightarrow \frac{1}{2m} \sum_{i=0}^m \left((wx^{(i)} + b) - y^{(i)} \right) x^{(i)}$$

$$= \frac{1}{m} \sum_{i=0}^m \left((wx^{(i)} + b) - y^{(i)} \right) x^{(i)}$$

$$\Rightarrow \boxed{\frac{1}{m} \sum_{i=0}^m \left(f_{wb}(x^{(i)}) - y^{(i)} \right) x^{(i)}}$$

$$\frac{\partial}{\partial b} J(w, b) = \frac{\partial}{\partial w} \left(\frac{1}{2m} \sum_{i=0}^m \left(f_{wb}(x^{(i)}) - y^{(i)} \right)^2 \right)$$

$$= \frac{\partial}{\partial w} \left(\frac{1}{2m} \sum_{i=0}^m \left((wx^{(i)} + b) - y^{(i)} \right) \right)$$

$$= \frac{1}{2m} \sum_{i=0}^m \left((wx^{(i)} + b) - y^{(i)} \right)$$

$$= \frac{1}{m} \sum_{i=0}^m \left(f_{wb}(x^{(i)}) - y^{(i)} \right)$$

Regression with multiple variables

Formula:-

$$f_{w,b}(\vec{x}) = w_1 x_1 + w_2 x_2 + w_3 x_3 \dots + b$$

$\vec{w} = [w_1, w_2, w_3 \dots w_m] \rightarrow$ vector } parameters
 b is a number

$$\vec{x} = [x_1, x_2, x_3 \dots x_m]$$

$$f_{w,b}(\vec{x}) = \vec{w} \cdot \vec{x} + b$$

dot product

$$\vec{w} \cdot \vec{x} = w_1 x_1 + w_2 x_2 + w_3 x_3 \dots + w_m x_m$$

vectorization:-

without vectorization:-

for j in range(16):

$$f = f + w[j] * x[j]$$

to $f = f + w[0] * x[0]$

t₁ $f = f + w[1] * x[1]$

⋮

t₁₅ $f = f + w[15] * x[15]$

with vectorization

$$w[0] \quad w[1] \quad \dots \quad w[15]$$

* * * *

$$x[0] \quad x[1] \quad \dots \quad x[15]$$

$$\begin{array}{c} \swarrow \quad \downarrow \quad \searrow \\ \boxed{w[0] * x[0]} + \boxed{w[1] * x[1]} + \dots + \boxed{w[15] * x[15]} \end{array}$$

for vectorization

$$\frac{\partial J}{\partial w} = \frac{1}{m} x^T \cdot \text{error} \quad \rightarrow \text{dot product}$$

$$\frac{\partial J}{\partial w} = \frac{1}{m} \sum \text{error} \rightarrow \frac{\text{sum of error}}{m}$$

eg:-

$$X = \begin{bmatrix} 1 & 2 \\ 2 & 3 \\ 3 & 4 \end{bmatrix} \quad \text{error} = \begin{bmatrix} 1 \\ -1 \\ 2 \end{bmatrix}$$

$$x^T \cdot \text{error} = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 3 & 4 \end{bmatrix} \cdot \begin{bmatrix} 1 \\ -1 \\ 2 \end{bmatrix} = \begin{bmatrix} 1(1) + 2(-1) + 3(2) \\ 2(1) + 3(-1) + 4(2) \end{bmatrix}$$

$$= \begin{bmatrix} 1 - 2 + 6 \\ 2 - 3 + 8 \end{bmatrix} = \begin{bmatrix} 5 \\ 7 \end{bmatrix}$$

$$x^T \cdot \text{error} / m = \begin{bmatrix} 5/3 \\ 7/3 \end{bmatrix} = \frac{\partial J}{\partial w} = dw$$

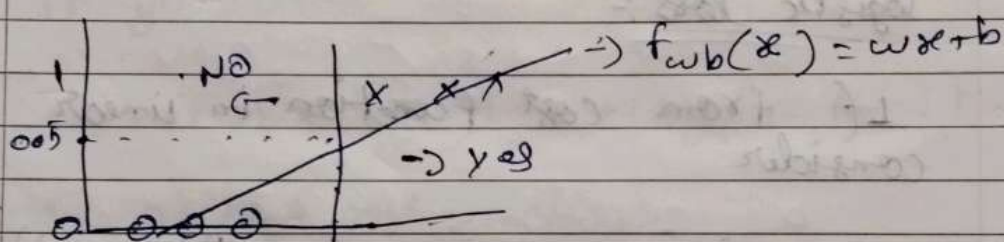
Classification:-

→ predicts categorical variables

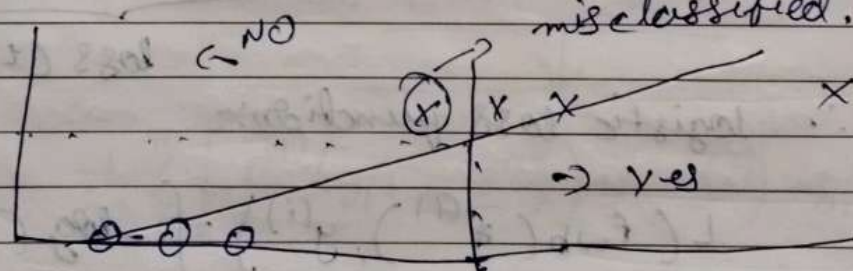
ex:	is this email spam?	Ans "y"	
		yes	no
		1	0
		True	False
		negative class	positive class

Why not to use linear regression for this?

ex- consider tumor size vs malignant.



→ not b/c consider an outlier misclassified.



logistic regression:-

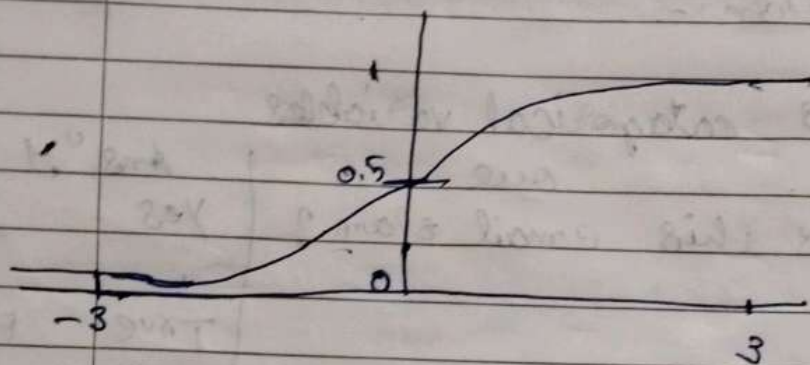
Sigmoid or logistic function:-

from linear reg:-

$$fwb(\bar{x}) = \underbrace{\bar{w} \cdot \bar{x} + b}_z$$

$$\therefore z = \bar{w} \cdot \bar{x} + b \Rightarrow fwb(\bar{x}) = \frac{1}{1 + e^{-z}}$$

$$\hookrightarrow \frac{1}{1 + e^{-(\bar{w} \cdot \bar{x} + b)}}$$



if $z = +ve$
we get answers
close 1, 0 for
-ve.

$$g(z) = \frac{1}{1 + e^{-z}}$$

logistic loss:-

If from cost function in linear reg consider

$$J(w, b) = \frac{1}{2m} \sum (f_{wb}(x^{(i)}) - y^{(i)})^2$$

loss (L) "consider"

∴ logistic loss function:-

$$L(f_{wb}(x^{(i)}), y^{(i)}) = \begin{cases} -\log(f_{wb}(x^{(i)})) & \text{if } y = 1 \\ -\log(1 - f_{wb}(x^{(i)})) & \text{if } y = 0 \end{cases}$$

simplified loss function

$$L(f_{wb}(\bar{x}^{(i)}, y^{(i)})) =$$

$$-y^{(i)} \log(f_{wb}(\bar{x}^{(i)})) - (1-y^{(i)}) \log(1-f_{wb}(\bar{x}^{(i)}))$$

if $y^{(i)} = 1$

$$L(f_{wb}(\bar{x}^{(i)}, y^{(i)})) = -1 \log(f(\bar{x}))$$

if $y^{(i)} = 0$

$$L(f_{wb}(\bar{x}^{(i)}, y^{(i)})) = -1 \log(1-f(\bar{x}))$$

simplified cost function

$$J(\bar{w}, b) = \frac{1}{m} \sum [L(f_{wb}(\bar{x}^{(i)}, y^{(i)}))]$$



$$J(\bar{w}, b) = \frac{1}{m} \sum_{i=1}^m [y^{(i)} \log(f_{wb}(\bar{x}^{(i)})) + (1-y^{(i)}) \log(1-f_{wb}(\bar{x}^{(i)}))]$$

Gradient descent for logistic regression

cost function

$$J(\bar{w}, b) = \frac{1}{m} \sum$$

repeat of

$$w_j = w_j - \alpha \frac{\partial}{\partial w_j} J(\bar{w}, b)$$

$$b = b - \alpha \frac{\partial}{\partial b} J(\bar{w}, b)$$

}

compute gradient:-

$$\frac{\partial J(\bar{w}, b)}{\partial w_j} = \frac{1}{m} \sum_{i=1}^m f_{wb}(\bar{x}^{(i)} - y^{(i)}) x_j^{(i)}$$

$$\frac{\partial J(\bar{w}, b)}{\partial b} = \frac{1}{m} \sum_{i=1}^m f_{wb}(\bar{x}^{(i)} - y^{(i)})$$

$\Rightarrow$ finally:-

repeat {

$$w_j = w_j - \alpha \left[\frac{1}{m} \sum_{i=1}^m f_{wb}(\bar{x}^{(i)} - y^{(i)}) x_j^{(i)} \right]$$

$$b = b - \alpha \left[\frac{1}{m} \sum_{i=1}^m f_{wb}(\bar{x}^{(i)} - y^{(i)}) \right]$$

} simultaneous update

why it is looking like linear reg?

$\rightarrow$ not actually because

<u>Linear regression</u>	<u>Logistic regression</u>
$f_{wb}(\bar{x}) = \bar{w} \cdot \bar{x} + b$	$f_{wb}(\bar{x}) = \frac{1}{1 + e^{-(\bar{w} \cdot \bar{x} + b)}}$

Regularization :-

→ its a technique to prevent overfitting by adding a penalty term to the loss function

cost function with regularization :-

(L2) Ridge :-
$$J(\bar{w}, b) = \underbrace{\frac{1}{2m} \sum_{i=1}^m (f_{wb}(\bar{x}^{(i)}) - y^{(i)})^2}_{\text{MSE}} + \underbrace{\frac{\lambda}{2m} \sum_{j=1}^n w_j^2}_{\substack{\text{regularization} \\ \text{term} \\ \text{or} \\ \text{Penalty}}}$$

⇒ λ = Tuning parameter

(c) Lasso :-

$$J(\bar{w}, b) = \frac{1}{2} \sum_{i=1}^m (f_{wb}(\bar{x}^{(i)}) - y^{(i)})^2 + \underbrace{\frac{\lambda}{2m} \sum_{j=1}^p |w_j|}_{\text{penalty}}$$

when to use?

Ridge	Lasso
→ lot of features, all features have small coeff	Small no of features, few features have large coeff

Regularized linear regression

$$\min_{w, b} J(\bar{w}, b) = \min_{w, b} \left[\frac{1}{2m} \sum_{i=1}^m (fwb(x^{(i)}) - y^{(i)})^2 + \frac{\lambda}{2m} \sum_{j=1}^m w_j^2 \right]$$

Gradient descent

repeat {

$$w_j = w_j - \alpha \frac{\partial}{\partial w_j} J(\bar{w}, b)$$

$$= \frac{1}{m} \sum_{i=1}^m (f(x^{(i)}) - y^{(i)}) x_j^{(i)} + \frac{\lambda}{m} w_j$$

$$b = b - \alpha \frac{\partial}{\partial b} J(\bar{w}, b)$$

→ no need to regularize b

} simultaneously update

Implementing gradient descent :-

$$w_j = w_j - \alpha \left[\frac{1}{m} \sum_{i=1}^m (f(x^{(i)}) - y^{(i)}) x_j^{(i)} + \frac{\lambda}{m} w_j \right]$$

$$b = b - \alpha \frac{1}{m} \sum_{i=1}^m (f(x^{(i)}) - y^{(i)})$$

$$w_j = \underbrace{w_j - \alpha \frac{\lambda}{m} w_j}_{w_j (1 - \alpha \frac{\lambda}{m})} - \alpha \frac{1}{m} \sum_{i=1}^m (f(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

$$w_j (1 - \alpha \frac{\lambda}{m})$$

→

$$\alpha \frac{\lambda}{m} = 0.01 \frac{1}{50} = 0.0002$$

$$w_j (1 - 0.0002) \text{ decrease it}$$

$$w_j (0.9998) \rightarrow \text{very small}$$

val in each iteration

Regularized logistic regression

$$J(w, b) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log(f(x^{(i)})) + (1 - y^{(i)}) \log(1 - f(x^{(i)})) \right] + \frac{\lambda}{2m} \sum_{j=1}^n w_j^2$$

Gradient descent:-

repeat of

$$w_j = w_j - \alpha \frac{\partial}{\partial w_j} J(w, b)$$

$$= -\frac{1}{m} \sum_{i=1}^m \left[(f(x^{(i)}) - y^{(i)}) x_j^{(i)} \right]$$

$$b = b - \alpha \frac{\partial}{\partial b} J(w, b)$$

$$+ \frac{\lambda}{m} w_0$$

}

Precision / Recall

lets take a simple binary classification of Disease.

$y=1$ if the person have disease or
 $y=0$ otherwise

Now we should make a confusion matrix

Predicted class	Actual class	
	1 True positive 15 (correct)	0 false positive 5 (ex)
0 false negative 10 (ex)		

Precision:-

out of all patients where we predicted $y=1$, what fraction actually have the disease?

$\Rightarrow$ True Positive

predicted positives

$\Rightarrow$ True Positive

True Pos + False Pos

$$= \frac{15}{15+5} = 0.75 \text{ (in this ex)}$$

Recall:- out of all patients that actually

have the disease what fraction did we correctly detect as having it?

$$\Rightarrow \frac{\text{True Positive}}{\text{True Positive} + \text{False Negative}} \Rightarrow \frac{\text{True Pos}}{\text{True Pos} + \text{False Neg}}$$

$$= \frac{15}{15 + 10} = \frac{15}{25} = 0.6$$

$\rightarrow$ So we can conclude that the model has 75% Precision, 60% Recall

F1 Score :-

Let's say we have 3 different algo's with different P, R on same data set. How can we choose them?

So we take F1 score

harmonic
 $\rightarrow$ mean

$$F_1 \text{ score} = \frac{1}{\frac{1}{P} + \frac{1}{R}} = 2 \times \frac{PR}{P+R}$$

Ex:

	P	R	F1 score	
Algo 1	0.5	0.4	0.444	✓ Pick this
Algo 2	0.7	0.1	0.175	
Algo 3	0.02	1.0	0.0392	

Decision trees

A decision tree is a supervised learning algorithm used for classification

→ let's take an example,
we are classifying cat vs dog using their features like

• ear shape → pointy

→ floppy

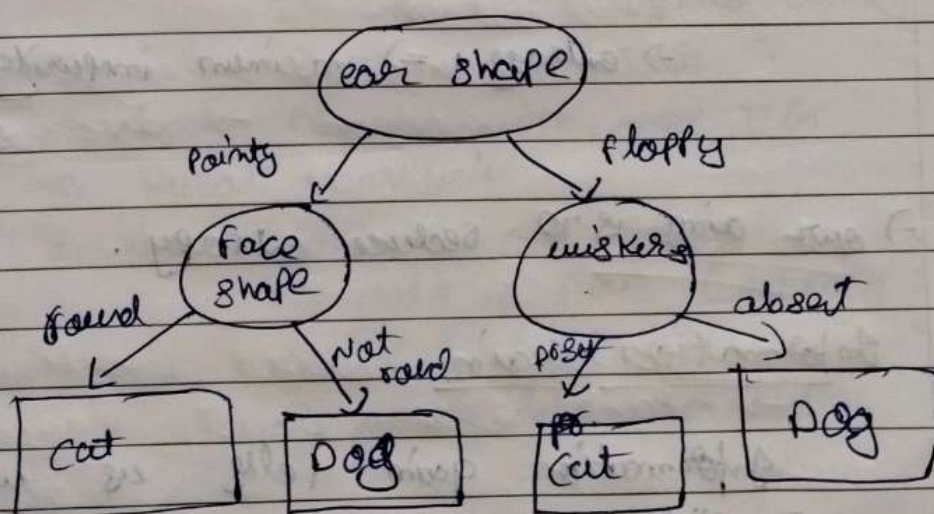
• face shape → round

→ not round

• whiskers → present

→ absent

Here we want to build a tree that classifies the data set



Here we have 3 features, but how to decide where to keep which feature at which node?

entropy :-

entropy measures randomness of data, it is denoted by 'H'

$$H(P_1) = -P_1 \log_2(P_1) - (1-P_1) \log_2(1-P_1)$$

Here :-

P_1 = in given labels how many are true labels

$$\Rightarrow \frac{\text{count}(y = 1)}{\text{len}(y)}$$

$\Rightarrow$ entropy can be 0 - 1

$\rightarrow$ entropy 0 $\Rightarrow$ pure node (all examples are same class)

$\rightarrow$ entropy 1 $\Rightarrow$ maximum impurity (50-50 split in class)

$\Rightarrow$ our aim is to reduce entropy.

Information Gain

Information gain tells us which node to split on

$$IG = H(P_i^{node}) - (w^{left} * H(P_i^{left}) + w^{right} * H(P_i^{right}))$$

where :-

$H(P_i^{node}) \Rightarrow$ entropy of root / parent node

$w^{left} \Rightarrow$ weight of left node
 $\hookrightarrow \frac{\text{no of samples in left node}}{\text{no of samples at parent node}}$

$w^{right} \Rightarrow$ (ii) vice versa

How decision tree works?

1. start with root node
2. calculate entropy of dataset
3. for each split :-
 - a) split data
 - b) compute weighted entropy
 - c) compute IG
4. choose feature with max IG
5. Repeat recursively
6. stop when :-
 - a.) node is pure (entropy = 0)
 - b.) no features left
 - c.) max dept reached

Random forest

- It is group (ensemble) of trees
- many trees trained on different random samples + random features
- final prediction = vote / average

* single DTree → overfits

Random forest →

reduce variance

gives stable prediction

How it works?

* dataset:-

→ create N samples

→ use sampling with replacement

* for each tree:-

→ at every split use random subset of features

* final output:-

classification → majority vote

Regression → average

XG Boost :-

- extreme gradient boosting
- each tree built sequentially
- each tree fixes previous tree's mistakes
- uses gradient descent on loss

Gradient idea (intuition)

1. fit tree rough prediction
2. compute error (gradient of loss)
3. next tree $\rightarrow$ predict err (work on error)
4. Repeat

Unsupervised learning

1. unsupervised learning :- learning from patterns from data with no labels

in supervised $\rightarrow (x \rightarrow y)$
(Bar shop $\rightarrow$ cat / dog)

in unsupervised :- $x \rightarrow$ Patterns / groups / structure

ex: grouping customers by behaviour
grouping news articles by topic
Anomaly detection

Clustering

$\rightarrow$ grouping similar data points together
 $\rightarrow$ no labels

Key Idea:-

points inside cluster

$\rightarrow$ close to each other

points across clusters

$\rightarrow$ far from each other

ex:

data $\rightarrow$ people

features $\Rightarrow x \rightarrow$ height

$y \rightarrow$ weight

clustering groups

$\rightarrow$ short - light, tall - heavy, mid - mid

K-Means

=> K-means groups data into K clusters using distance from centroids

K = no. of clusters

How it works?

1. choose K :-

K = 3 (3 clusters)

2. Initialize centroids

-> Randomly pick K points

-> these are cluster centroids

3. Assign points to nearest centroid

-> for every point find the distance between the point and $C_1, C_2, C_3 \dots C_K$

-> Assign the point to closest centroid

4. update centroids

-> new centroid = mean of all points in that cluster

* -> do it for every feature

->

$$\text{centroid} = (x_1 + x_2 + x_3 + \dots + x_n) / n$$

-> * x is a point

5. Repeat

→ Repeat steps 3, 4 until

1. centroids stop changing
or

2. changes are very small

Cost function

$$J(c^{(1)}, \dots, c^{(m)}, \mu_1, \dots, \mu_K) = \frac{1}{m} \sum_{i=1}^m ||x^{(i)} - \mu_{c(i)}||^2$$

$c^{(i)}$ = index of cluster (1, 2, ..., K) to which sample $x^{(i)}$ is currently assigned

μ_K = cluster index K

$\mu_{c(i)}$ = centroid of cluster to which sample $x^{(i)}$ has been assigned

⇒ $\left(\frac{1}{m} \sum_{i=1}^m ||x^{(i)} - \mu_{c(i)}||^2 \right)$

taking Average

→ Squared distance b/w point, its centroid

⇒ J = how bad the cluster is
distance = error
mean = centroid
goal = to minimize J

Algo:-

Repeat of

assign points to cluster

for $i=1$ to m :

$c(i) :=$ index of cluster

centroid closest to $x(i)$

move cluster centroid

for $k=1$ to K

$\mu_k :=$ average of points in cluster k

y

Anomaly detection

→ used for finding rare, unusual or abnormal data points

Ex:

- * credit card fraud
- * server error log
- * manufacturing logs, etc

Idea:-

Normal data → clustered / dense

abnormal data → far away / lonely

Methods:-

1. Statistical:-

- assume data follows distribution
- gaussian / normal distribution
- use probability

2. distance based

- far from distance neighbours

3. density based

- low density region → anomaly

4. ML models

- Isolation forest
- one-class SVM
- auto encoders

Gaussian anomaly (most common):-

- this is totally based on probability
- uses gaussian distribution.

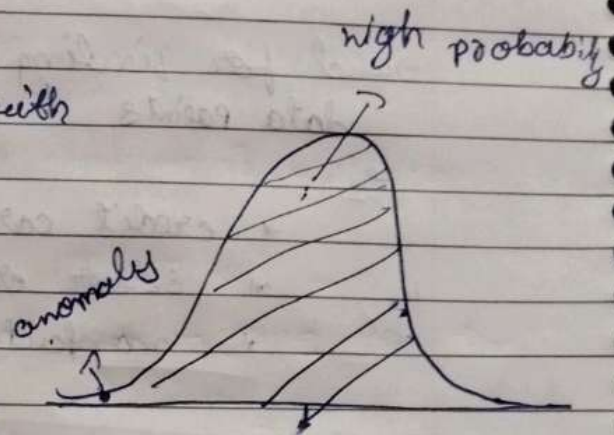
steps:-

→ create model with original data points

→ this computes probability

→ if a given data point

is greater than threshold value its not "anomaly"



$$\Rightarrow P(x) < \epsilon \rightarrow \text{anomaly}$$

epsilon (threshold)

formula:-

$$P(x) = \text{gaussian}(\mu, \sigma^2)$$

$$\Rightarrow P(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$



for 1 feature

$$\Rightarrow P(x) = \prod_{j=1}^n P(x_j; \mu_j, \sigma_j^2)$$



$$P(x) = \prod_{j=1}^n \left[\frac{1}{\sqrt{2\pi\sigma_j^2}} e^{-\frac{(x_j - \mu_j)^2}{2\sigma_j^2}} \right]$$

1, multiple features

Intuition:-

→ for each feature → gives normality score

↓
"have normal the point is"

→ multiply all → overall score

→ if final $P(x)$ very small → anomaly

Multivariate gaussian

→ used when features are co-related

Formula:-

$$P(x) = \frac{1}{(2\pi)^{n/2} |\Sigma|^{1/2}} e^{-\frac{1}{2}(x-\mu)^T \Sigma^{-1}(x-\mu)}$$

where:

Σ → covariance matrix

$|\Sigma|$ → det of Σ

Σ^{-1} = inverse of Σ

transpose

$$(x-\mu)^T \Sigma^{-1} (x-\mu)$$

↳ Mahalanobis distance

Note:-

→ Multi-feature $\neq$ multivariate

difference:-

multiple:-

1. multiply each feature probability
2. independent features

Multivariate:-

1. single big matrix
2. correlated features